Cripto II

by: mago del cobre



Universidad Rey Juan Carlos

Universidad Rey Juan Carlos

Cripto



Tu cara al ver que toca cripto



Lo que veremos hoy

- XOR (again)
- AES
- AES-ECB
- AES-CBC
- Módulo (again)
- RSA

(y obviamente ataques a cada cifrado)



XOR

Universidad
Rey Juan Carlos

Universidad Rey Juan Carlos

XOR (again baby)

XOR es un operador bit por bit que devuelve 0 si son iguales y devuelve 1 si son diferentes, se representa con ⊕ pero en mayoría de casos con ^.

A	B	Output
0	0	0
0	1	1
1	0	1
1	1	0

Universidad Rey Juan Carlos

Simple no?

Propiedades

- **Conmutativa: A ⊕ B = B ⊕ A**
- **Asociativa: A ⊕ (B ⊕ C) = (A ⊕ B) ⊕ C**
- **Identidad: A ⊕ 0 = A**
- **Auto-inversa: A ⊕ A = 0**

Lo importante es que

Plaintext ⊕ key = Cypher -> Cypher ⊕ key = plaintext

Demostración:

(Plaintext ⊕ key) ⊕ key = Plaintext ⊕ (key ⊕ key) = Plaintext ⊕ 0 = Plaintext

(hacer los retos de xor de repaso del ctf del viernes pasado)



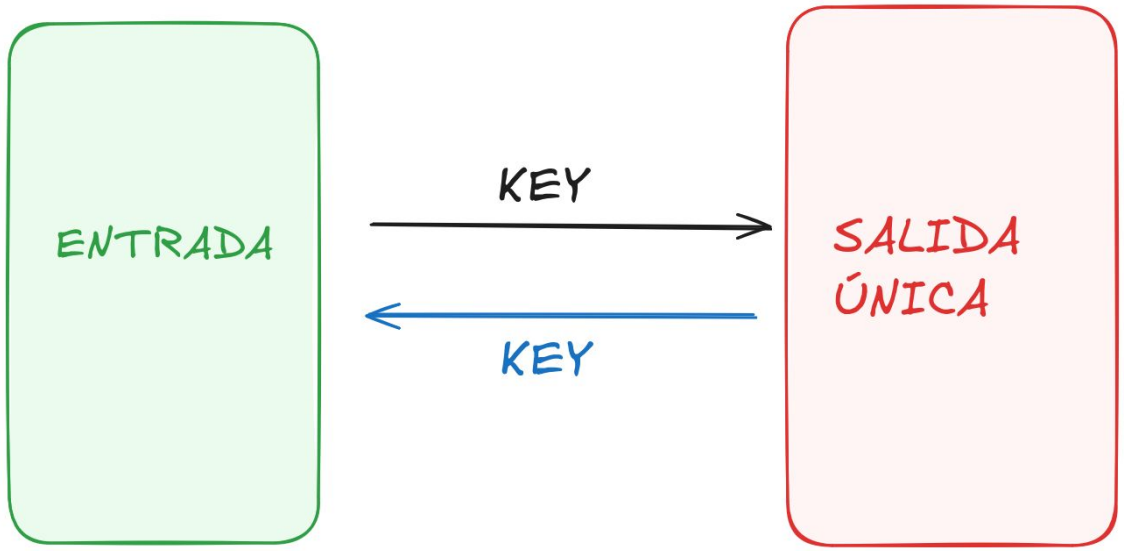
AES

Universidad
Rey Juan Carlos

Universidad Rey Juan Carlos

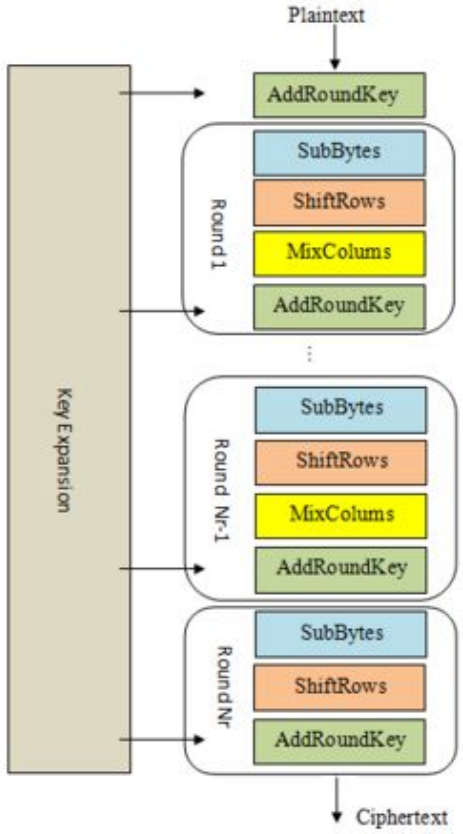
AES

AES es un sistema de cifrado por bloques que hace una permutación con clave. Que significa? Que para cada bloque de entrada hay un bloque único de salida, con una llave que indica que permutación hacer.



¿Como se consigue esto?

Cómo funciona AES-128?

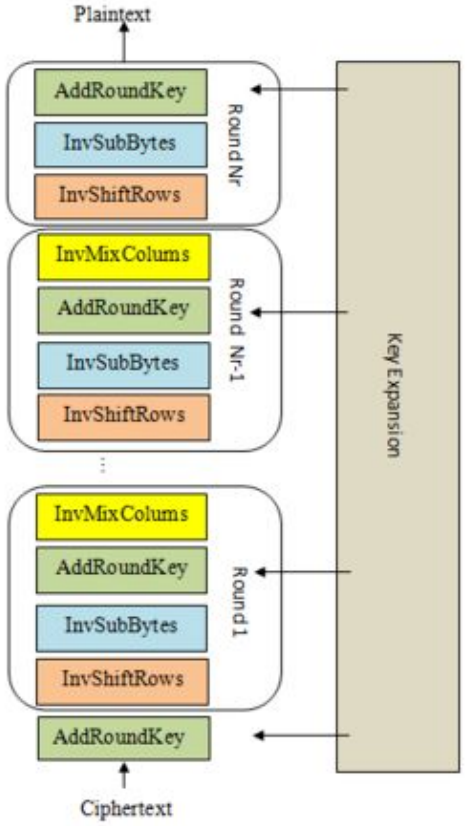


¿Cómo funciona AES simplificado?

AES no codifica un texto plano si no que coje los 16 bytes de entrada y los divide en una matriz 4x4 llamada estado.

Usando el video de referencia de abajo, estas son las fases del cifrado AES:

AES empieza cifrando una llave de 128 bits en 11 bloques, a esto se le llama el key expansion.

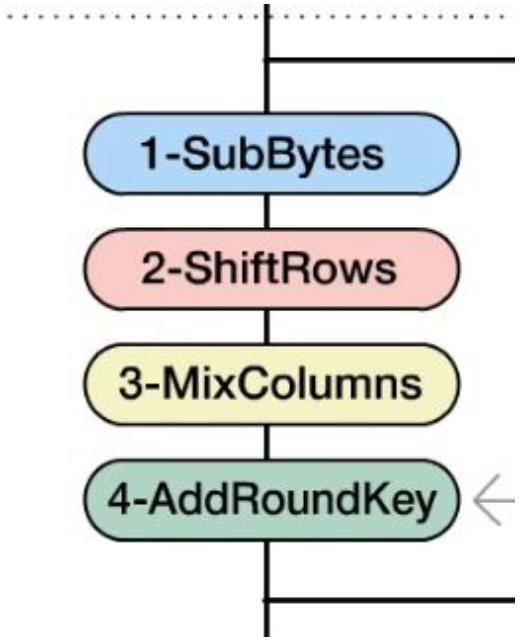


https://www.youtube.com/watch?v=gP4PqVGudtg

Universidad Rey Juan Carlos

Cómo funciona AES?

1) SubBytes: Cada byte del estado se sustituye por un byte diferente según una tabla de consulta (*S-box*).
2) ShiftRows: Las últimas tres filas de la matriz de estado se desplazan (circularmente a la izquierda) una, dos o tres columnas.
3) MixColumns: Se realiza una multiplicación matricial sobre las columnas del estado, combinando los cuatro bytes de cada columna. Este paso se omite en la ronda final.
4) AddRoundKey: Se realiza una operación XOR entre los bytes de la subclave de la ronda actual y los bytes del estado.

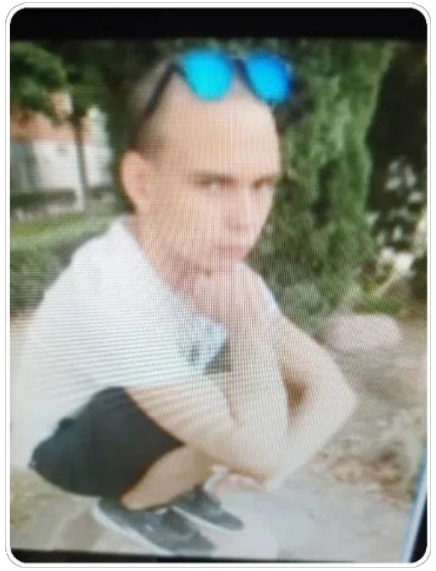


Universidad Rey Juan Carlos

Lo importante

LO IMPORTANTE ES LO SIGUIENTE

Esto cifra 16 bytes, pero un mensaje es más largo, entonces ¿Cómo lo hacen para cifrar 128 bytes?





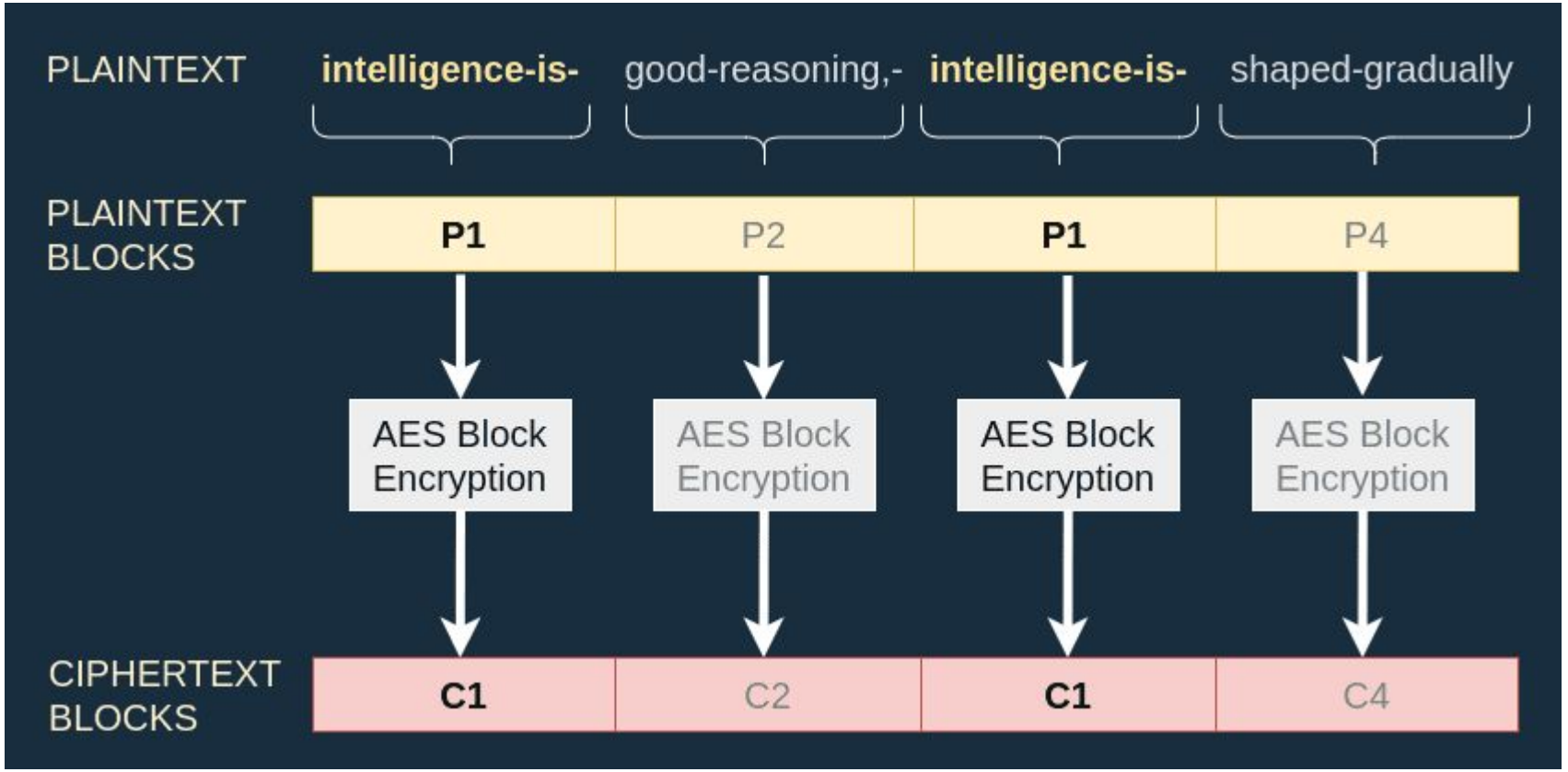
MODOS DE OPERACIÓN
ECB y CBC

Universidad
Rey Juan Carlos

Universidad Rey Juan Carlos

ECB (Estoy Cockeando Bien)

La respuesta es: usar AES en ECB mode
ECB lo que hace es dividir el texto en bloques y encriptarlos usando AES (como si fuese una caja mágica que encripta las cosas) de manera INDEPENDIENTE.

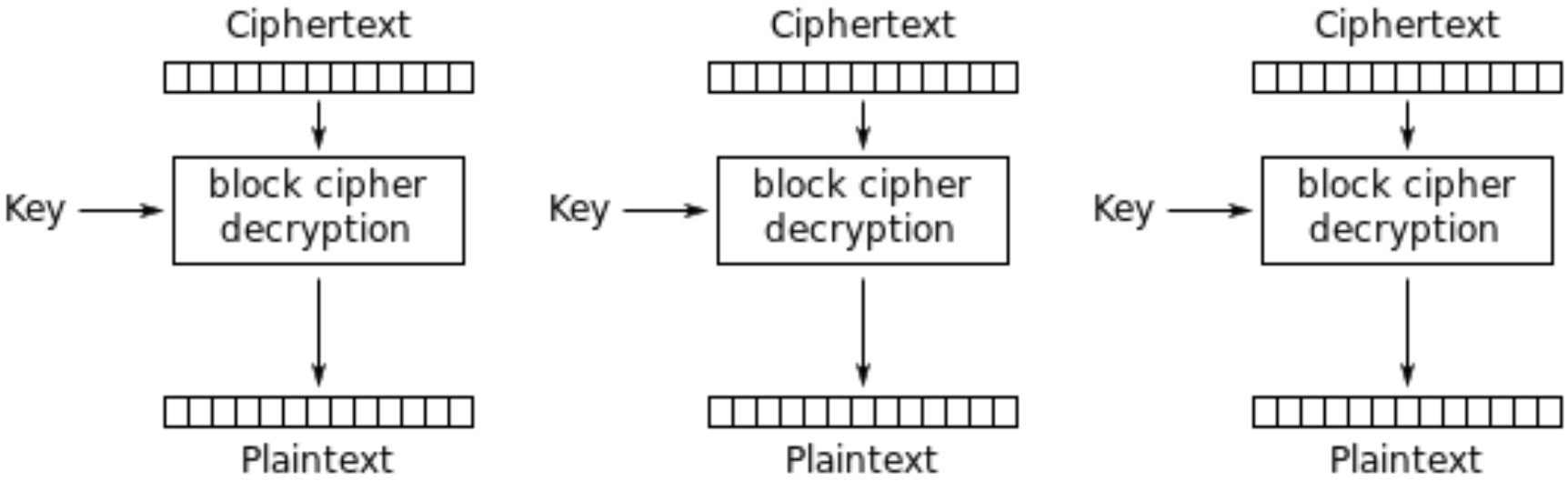


https://braincoke.fr/write-up/cryptopals/cryptopals-detect-aes-in-ecb-mode/

Universidad Rey Juan Carlos

ECB (Estoy Cockeando Bien)

Descifrado



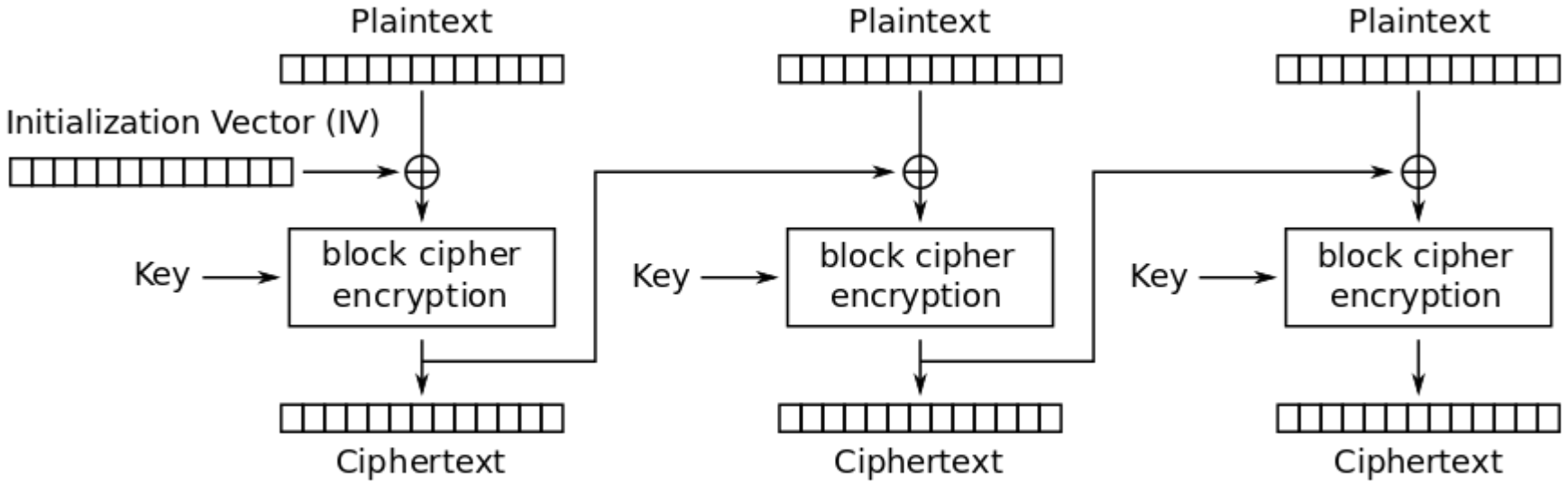
Electronic Codebook (ECB) mode decryption

https://wiki.bi0s.in/crypto/aes-ecb/

Universidad Rey Juan Carlos

AES No solo tiene 1 modo

CBC es parecido al **ECB** pero el cifrado depende del anterior bloque. Cada bloque es XOReado con el anterior bloque, y el primer bloque es XOReado por 16 bytes aleatorios IV (Initialization Vector)



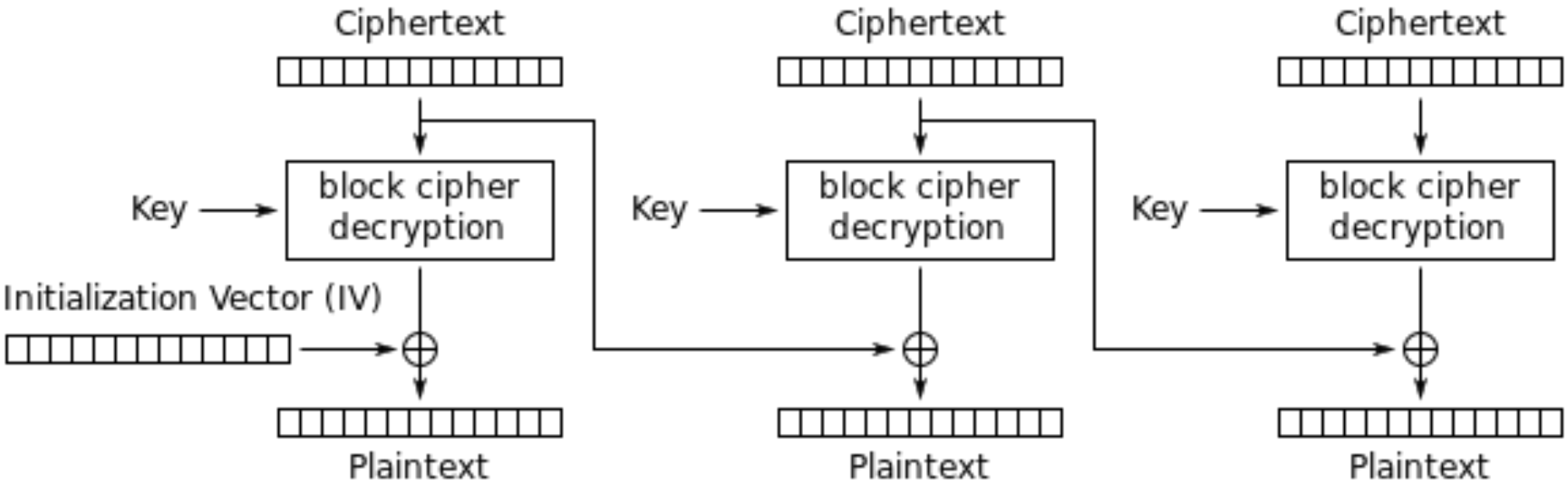
Cipher Block Chaining (CBC) mode encryption

https://wiki.bi0s.in/crypto/aes-cbc/

Universidad Rey Juan Carlos

CBC (Como Bocata de Cangrejo)

Descifrado



Cipher Block Chaining (CBC) mode decryption

https://wiki.bi0s.in/crypto/aes-cbc/

Universidad Rey Juan Carlos

PROBLEMAS

El ECB cifra cada bloque por separado usando la misma clave, el problema?

¿Pues qué pasa si hay 2 textos iguales?

Bloque A = Bloque B -> AES_ECB(A) = AES_ECB(B)

sus correspondientes textos cifrados son idénticos, esto revela datos sin necesidad de romper matemáticamente el algoritmo del AES.

ECB VS CBC

ECB vs. CBC



Original



ECB



CBC

https://wiki.bi0s.in/crypto/aes-cbc/

Universidad Rey Juan Carlos

Ataque a ECB, one bit at a time

¿Cómo aprovechamos la vulnerabilidad?

Imaginemos que hay un servidor que antes de cifrar concatena tu entrada con un secreto (la flag):

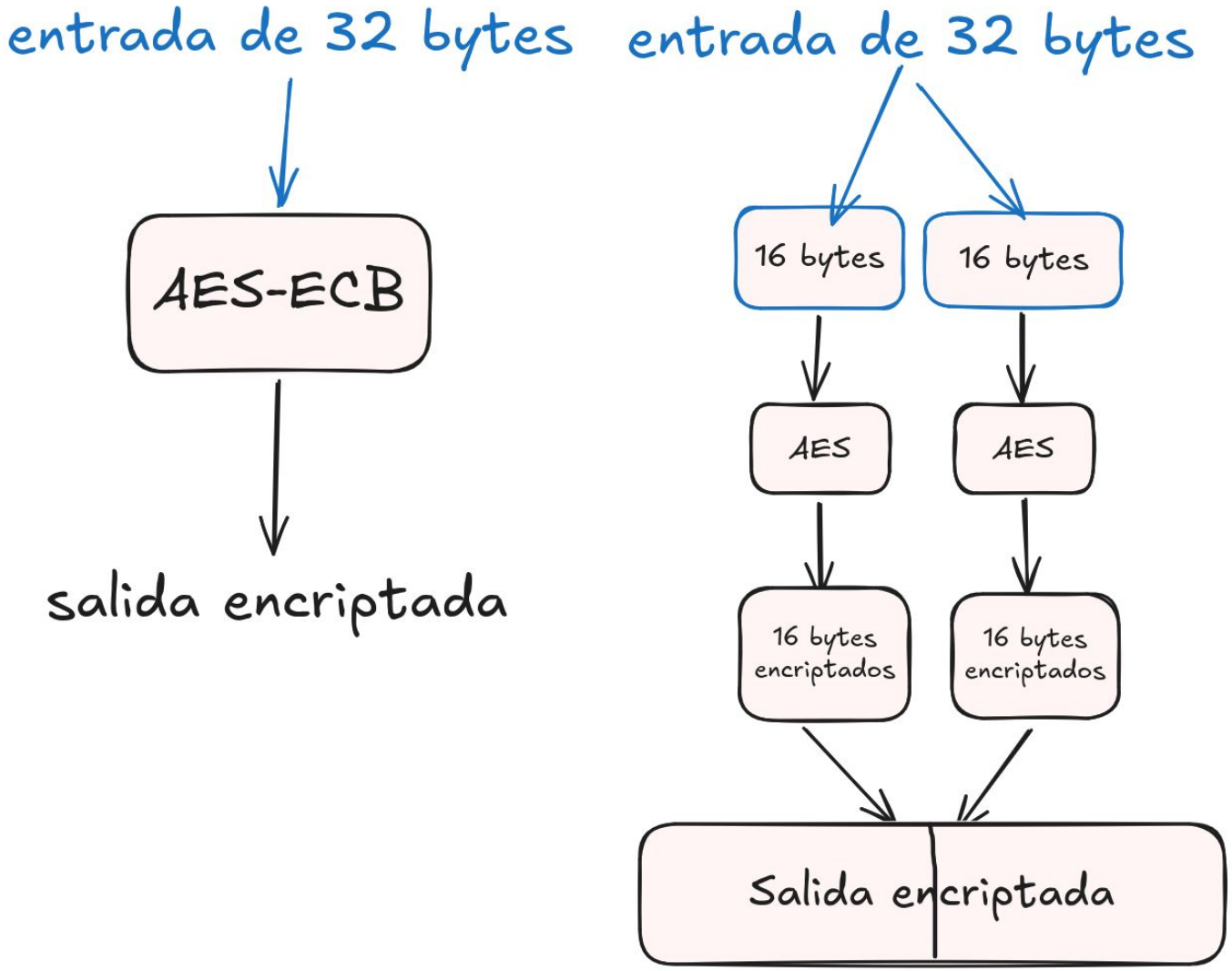
AES_ECB(entrada || flag)

¿Cómo sacamos la flag? Pues con un poco de fuerza bruta jeje



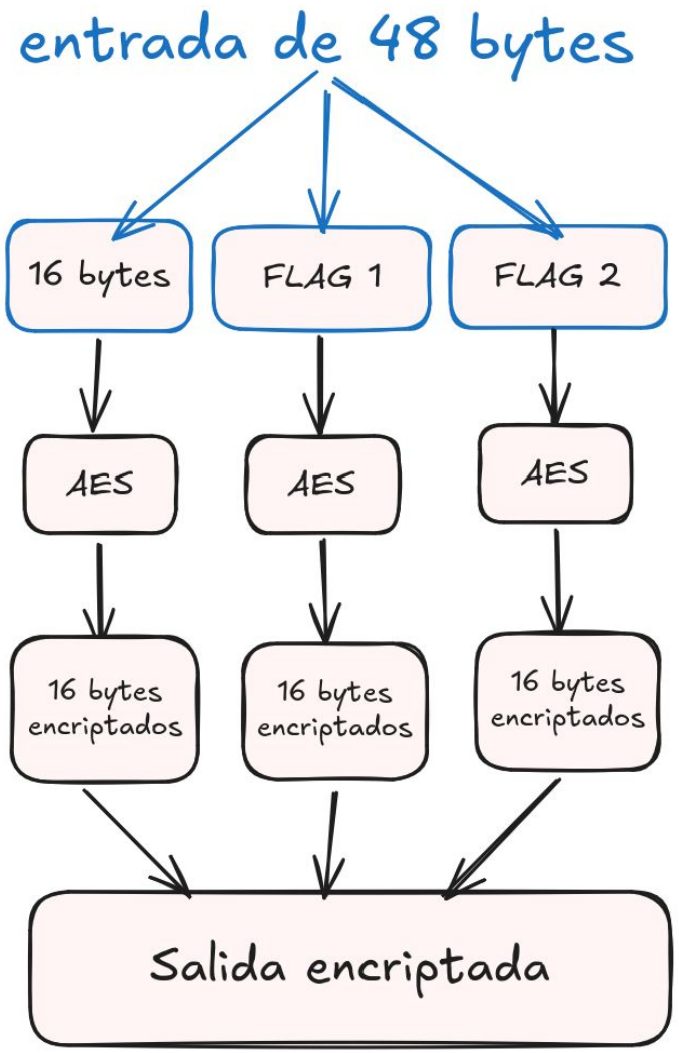
Universidad Rey Juan Carlos

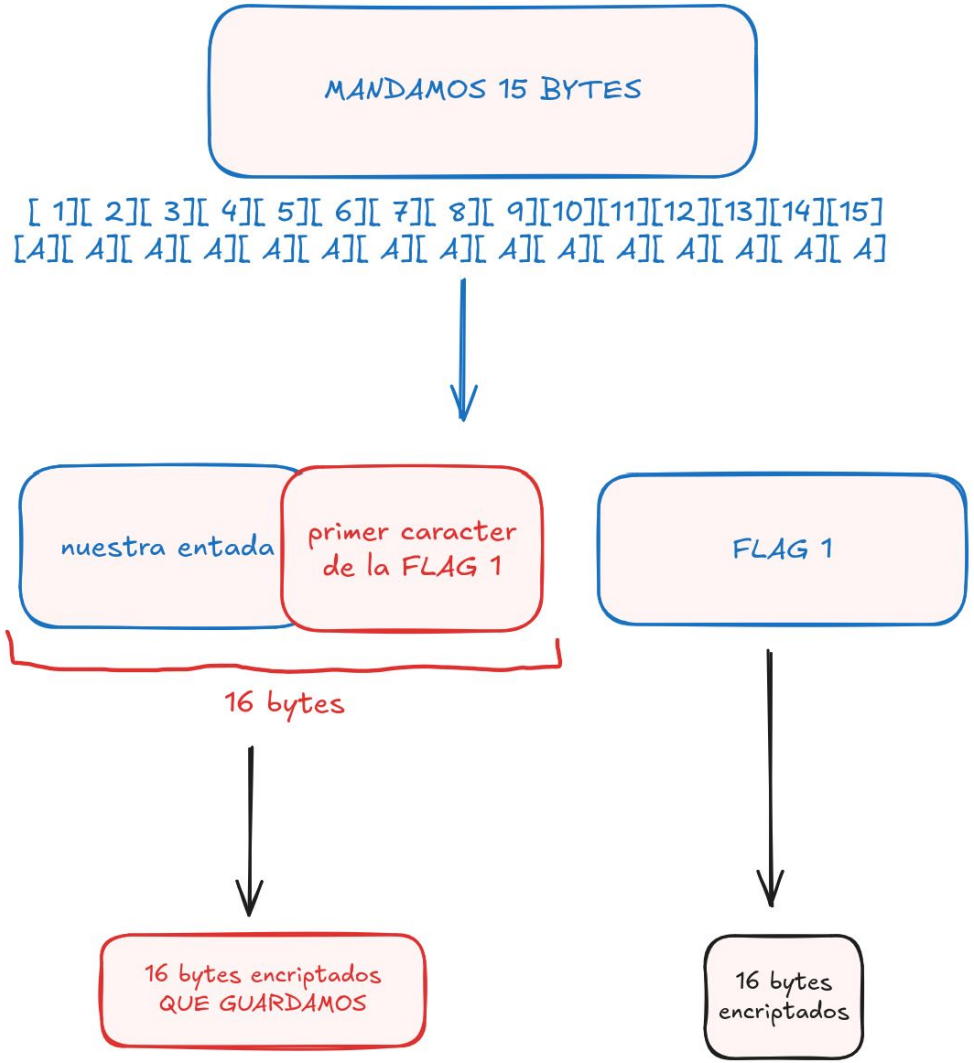
Idea del ataque



Universidad Rey Juan Carlos

Idea del ataque





Universidad Rey Juan Carlos

Idea del ataque

SCRIPT EN PYTHON

Mandamos encriptar 15 A mas un caracter X, este caracter sera desde 'a' hasta 'Z' pasando por los numeros. Lo llamaremos Bloque A

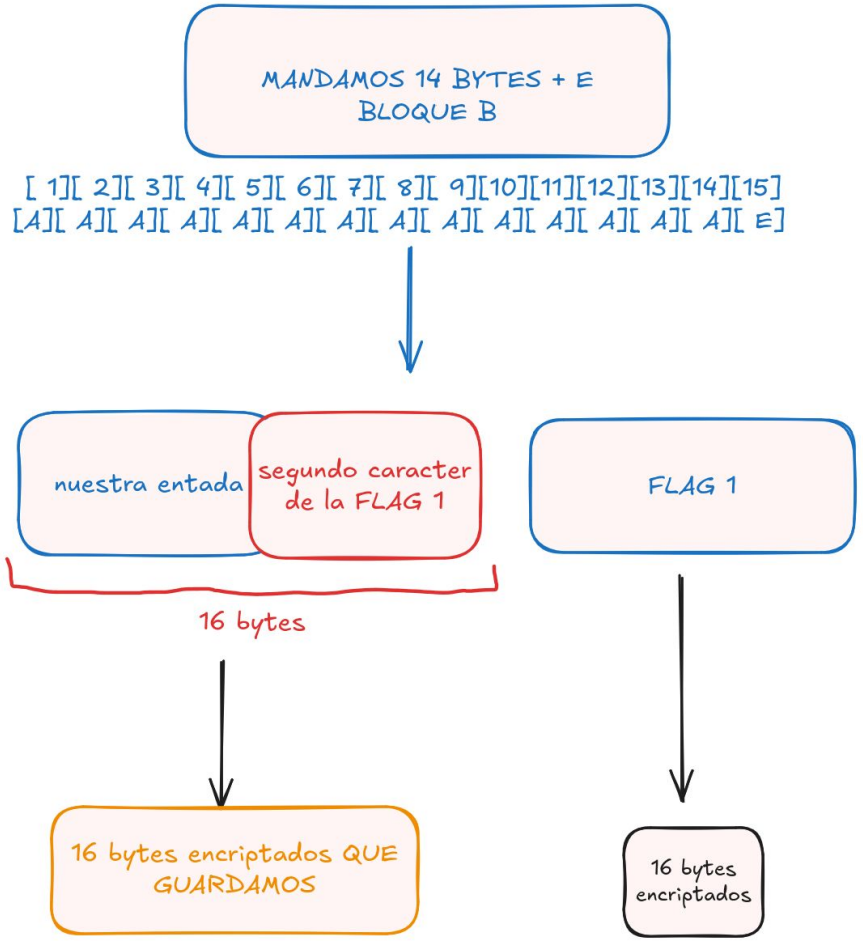
Bloque A = 16 bytes encriptados QUE GUARDAMOS

FLAG = E

Mandamos encriptar 14 A + E mas un caracter X, este caracter sera desde 'a' hasta 'Z' pasando por los numeros. Lo llamaremos Bloque B

Bloque B = 16 bytes encriptados QUE GUARDAMOS

...



Universidad Rey Juan Carlos

Script en python

```python
flag = b""

# Bucle para descubrir los primeros 16 caracteres (el primer bloque de la FLAG)
for i in range(16):

    # El relleno o padding baja de 1 en 1 en cada iteración: 1º- 15 "A", luego 14 "A", luego 13 "A"...
    # Esto aísla siempre una nueva letra desconocida en la posición 16 del bloque
    padding = b"A" * (15 - i)

    # Obtenemos cómo cifra el servidor ese bloque (nos quedamos los primeros 16 bytes)
    # Oraculo es como he llamado a la funcion que encripta en vuestro caso sera con nc y pwntools.
    bloque_objetivo = oraculo(padding)[0:16]

    # Probamos los 256 caracteres posibles
    for j in range(256):
        letra_adivinada = bytes([j])

        # Creamos un bloque con el payload, que es Padding + Letras que ya sabemos + Letra a probar
        payload = relleno + flag + letra_adivinada
        bloque_prueba = oraculo(payload)[0:16]

        # Si nuestro bloque coincide con el del servidor, acertamos la letra
        if bloque_prueba == bloque_objetivo:
            flag += letra_adivinada
            print(flag_descubierta)
            break # se sale de este bucle (j in range(256)) para la siguiente iteración
```

Universidad Rey Juan Carlos

Script en python

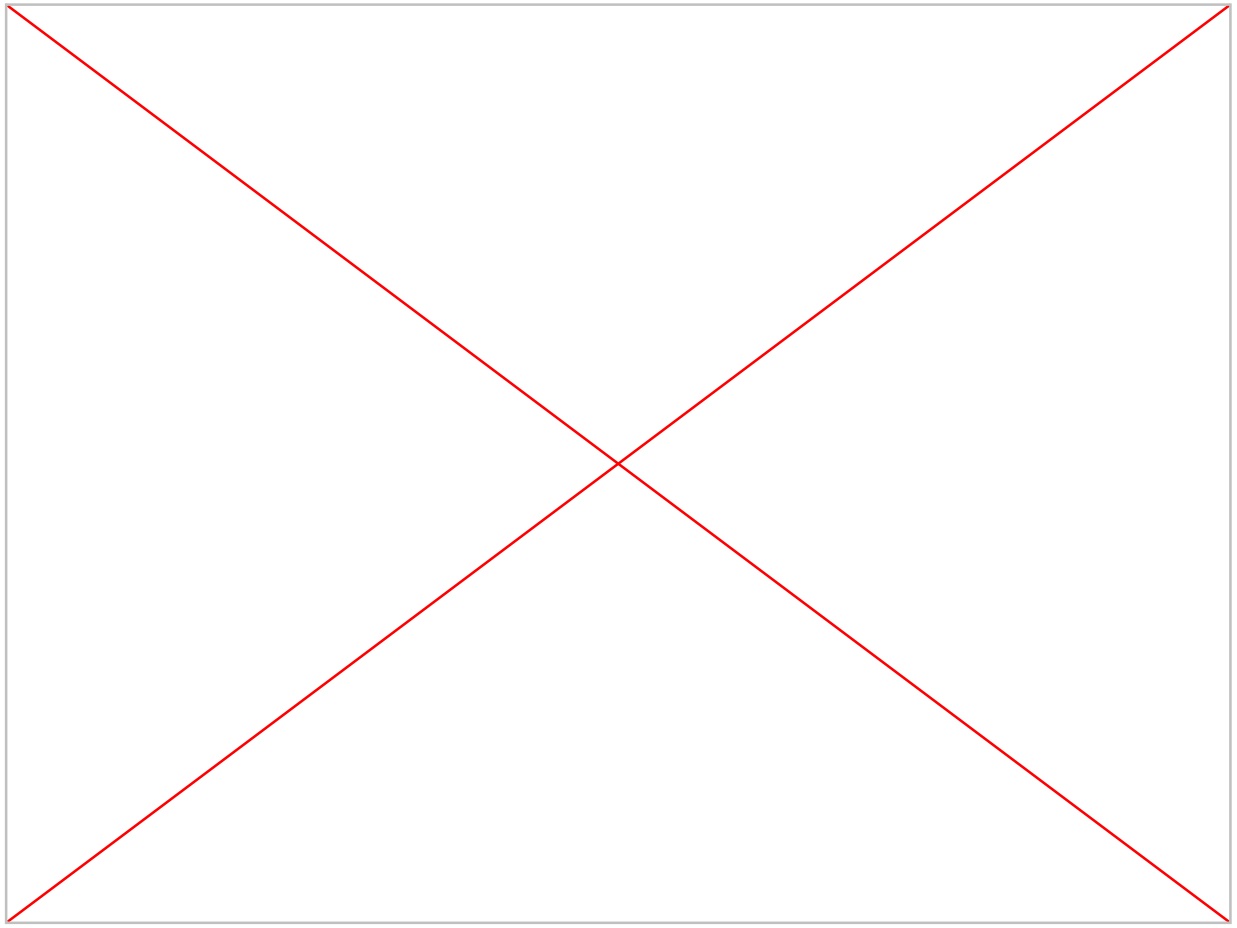
```
CHARSET = (
    string.printable.encode()
    .replace(b"\n", b"")
    .replace(b"\r", b"")
    .replace(b"\x0b", b"")
    .replace(b"\x0c", b"")
)
```

Este CHARSET es una variable de python que hemos definido que evita caracteres problemáticos, así el script funciona sin errores

Universidad Rey Juan Carlos

Ataque a ECB, one bit at a time

¿Cómo aprovechamos la vulnerabilidad?



Universidad Rey Juan Carlos



Lo habéis entendido?
a hacer el retillo

Universidad Rey Juan Carlos



MÓDULO (again)

Universidad Rey Juan Carlos

Universidad Rey Juan Carlos

Modular exponation

El módulo es el resto de la división entera
15 mod 12 = 3

La cuestión es que en RSA se usa módulo exponencial, esto qué significa? Pues:

$$2^{10} \mod 17$$

En python: `pow(base, exponent, modulus)`

Esto nos sirve para crear una **trapdoor function**.
Una **trapdoor function** es una función que es fácil de computar, pero MUY difícil de invertir sin la información necesaria.

Universidad Rey Juan Carlos

El inverso del modulo

El inverso de un módulo es:
El inverso multiplicativo modular de un número **A** respecto a un módulo **M** es encontrar un entero **X** que satisfaga la siguiente congruencia:

$$A \cdot X \equiv 1 \bmod(M)$$

Es obligatorio que A y M sean coprimos (su máximo común divisor es 1)

```
En python: pow(base,-1, modulus)
```



RSA criptografía asimétrica

Universidad Rey Juan Carlos

Universidad Rey Juan Carlos

RSA

RSA cómo funciona?



bob



alice

RSA

RSA cómo funciona?



yo what up



bob

alice

RSA

RSA cómo funciona?



bob



alice

RSA

RSA cómo funciona?



bob



alice

RSA

RSA cómo funciona?



bob



alice

RSA

RSA cómo funciona?



bob



alice

RSA

RSA cómo funciona?

Pues como bien ha dicho alice, cualquier persona tiene un par de llaves.

La llave pública la cual conoce todo el mundo, está SOLO se usa para encriptar los mensajes.

La llave privada la cual solo conoce alice y sirve para descifrar el mensaje privado.

De ahí criptografía asimétrica, pero como se hace?



alice

Universidad Rey Juan Carlos

RSA public key

La encriptación RSA es la modular exponetion de un mensaje con un módulo ***N*** y un exponente ***e.***

$$(Mensaje)^e \bmod N$$

El módulo N viene dado de un producto entre 2 números primos MUY grandes llamados p y q.

$$N = p \cdot q$$

El exponente ***e*** es también un número muy elevado, comúnmente 0x10001 = 65537.

Para que? Pues para crear una trapdoor function, una función muy difícil de invertir si no sabes la información necesaria.

Universidad Rey Juan Carlos

RSA public key

La seguridad de RSA se basa en la dificultad de factorizar N, si los primos pueden ser deducidos, podemos calcular la función toniente de Euler y así desencriptar el mensaje.

$$\phi(N)$$

Esta función básicamente es “De todos los números enteros del 1 hasta N, ¿Cuántos de ellos NO comparten ningún divisor con N (salvo el 1)?”
Estos números se llaman coprimos.
Vale, pero ¿Porque esta función rompe la seguridad de RSA?

Universidad Rey Juan Carlos

Euler

Resulta que esta función cuando N está compuesto por un producto de dos primos hace el siguiente cálculo:

$$\phi(N) = (p - 1) \cdot (q - 1)$$

vale, pero ¿PARA QUÉ SIRVE ESTA FUNCIÓN?
¿Porque esta función rompe la seguridad de RSA?

Universidad Rey Juan Carlos

RSA private key

La clave privada sirve para descifrar los textos encriptados con su respectiva clave pública. Esta es la llave que nos permite invertir la trapdoor function de manera fácil. La llave privada es:

$$d \equiv e^{-1} \bmod \phi(N)$$

En python: `pow(e,-1, ϕ(N))`

¿Porque esta función rompe la seguridad de RSA?
Porque podemos desencriptar el mensaje fácilmente sabiendo la función

Universidad Rey Juan Carlos

RSA private key

Para descifrar es simplemente

$$(Mensaje\ cifrado)^{d} \bmod N$$

y con esto es todo RSA (salvo las firmas… y ataques…)

Universidad Rey Juan Carlos

Ataques RSA

Hacer RSA sin python: https://www.dcode.fr/chiffre-rsa

N débil: https://factordb.com/

Debilidad en el Exponente e: el M es pequeño de modo que $M^x < N$

otros ataques:
https://www.ellaberintodefalken.com/2014/09/ataques-debilidades-rsa.html
https://hacktricks.wiki/es/crypto/public-key/rsa/index.html

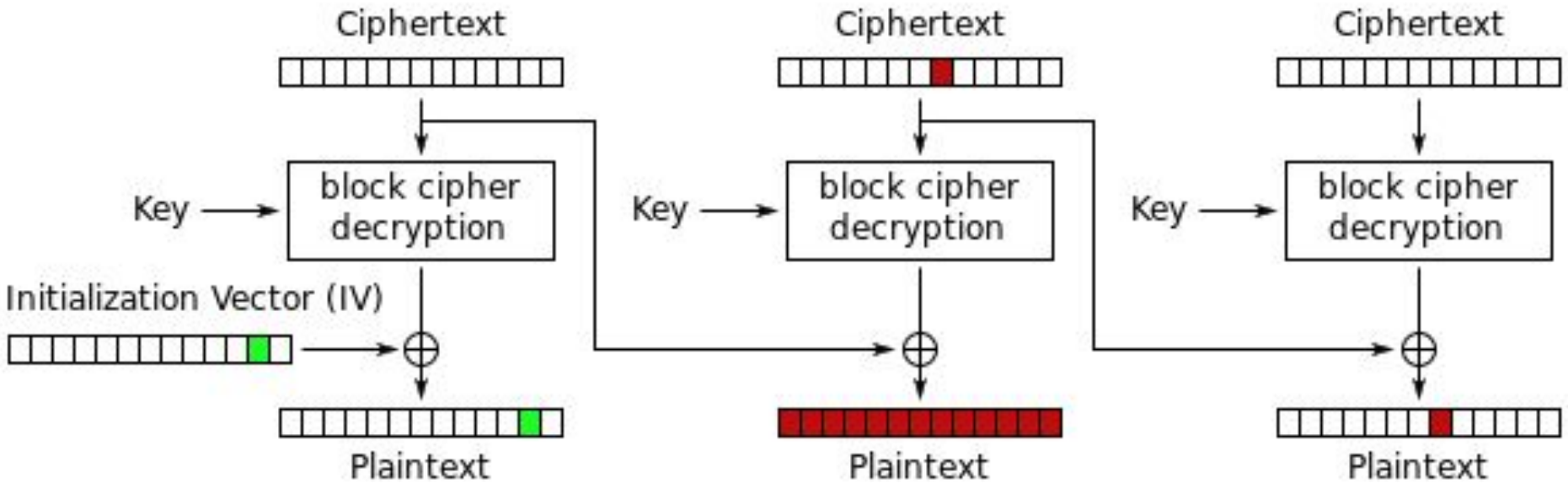


CBC bit flipping

Universidad
Rey Juan Carlos

Universidad Rey Juan Carlos

CBC bit flipping



Cipher Block Chaining (CBC) mode decryption

https://crypto.stackexchange.com/questions/66085/bit-flipping-attack-on-cbc-mode